

IoT : Intergiciels de Messaging

Thierry Desprats

Faculté des Sciences et d'Ingénierie
Département Informatique – Laboratoire IRIT
(Prénom.Nom@irit.fr)

IoT : Intergiciels de *Messaging*

1 – Intergiciels orientés « Réseau »

Intergiciels : définition

- « **Middleware** » est un anglicisme
 - Intergiciel
 - Logiciel de médiation
- Définition générale :
 - Un **intergiciel** est un logiciel tiers qui agit comme une passerelle intermédiaire entre divers autres composants comme des applications (hétérogènes), des outils ou encore des bases de données tout en apportant des services unifiés.
 - Rôle de ciment, de glu, de substrat logiciel, de médiation, de bus d'échange, de plateforme d'intégration...
 - Souvent les intergiciels sont construits sur des protocoles et des formats de représentation unifiés standardisés, proposent des APIs, des outils et des services utiles à la fois en phases de développement, de déploiement et d'exécution d'un écosystème composite et distribué :
= = > **rôle d'intégration**

Intergiciels « réseaux » : définition

- **Network-Centric Middlewares (NCM) :**

- Architectures intergicielles **orientée Transport**, véritables infrastructures techniques de médiation entre composants, permettant d'effectuer le transfert de messages d'une application vers une autre; plus qu'un protocole... !

- Cibles :

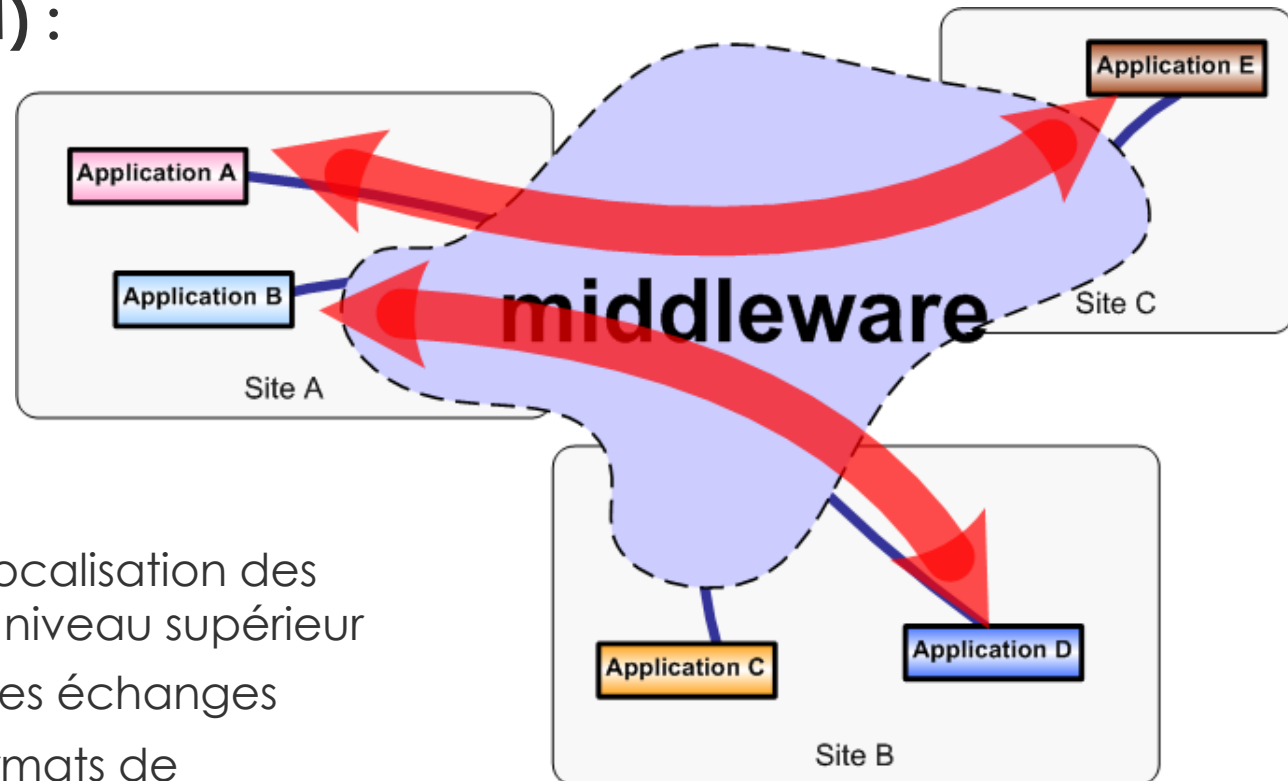
- Propagation des données : **MOMs** (*Message Oriented Middleware*)
==> modalité d'échange asynchrone, à base de messages, basé sur une communication indirecte

- Intégration des traitements : **ORBs** (*Object Request Broker*)
==> Sémantique d'invocation plutôt synchrone (possiblement asynchrone) point-à-point de fonctions, de méthodes, d'opérations...

- ==> **Dynamique des flux et des traitements**

Intergiciels « réseaux » : définition

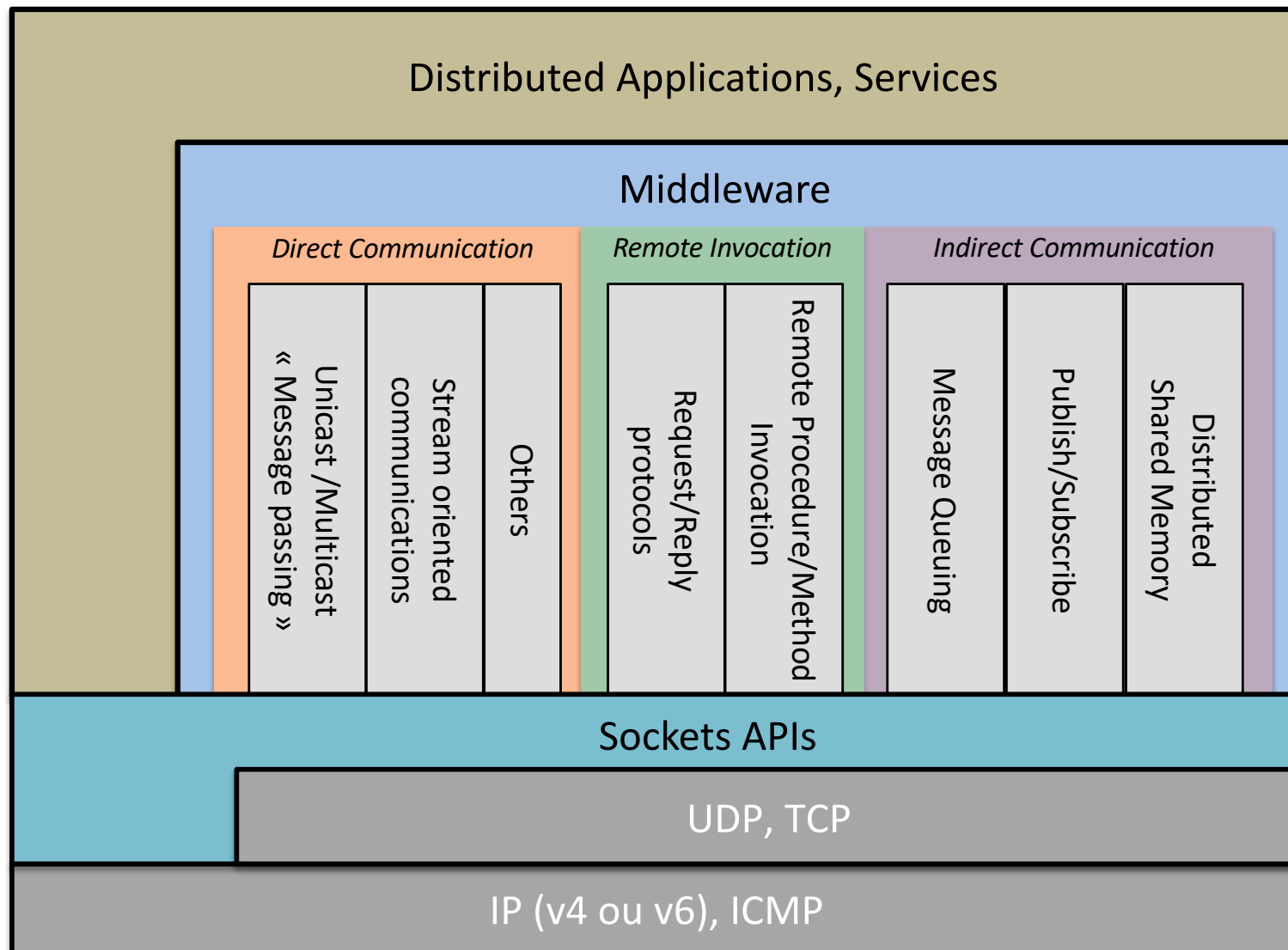
■ Illustration (NCM) :



■ Services offerts :

- Identification et localisation des applications à un niveau supérieur
- Acheminement des échanges
- Conversion de formats de représentation (interopérabilité)
- Fonctions additionnelles de sécurité, de routage sophistiqué, d'équilibrage de charge, de tolérance aux pannes, de QoS...

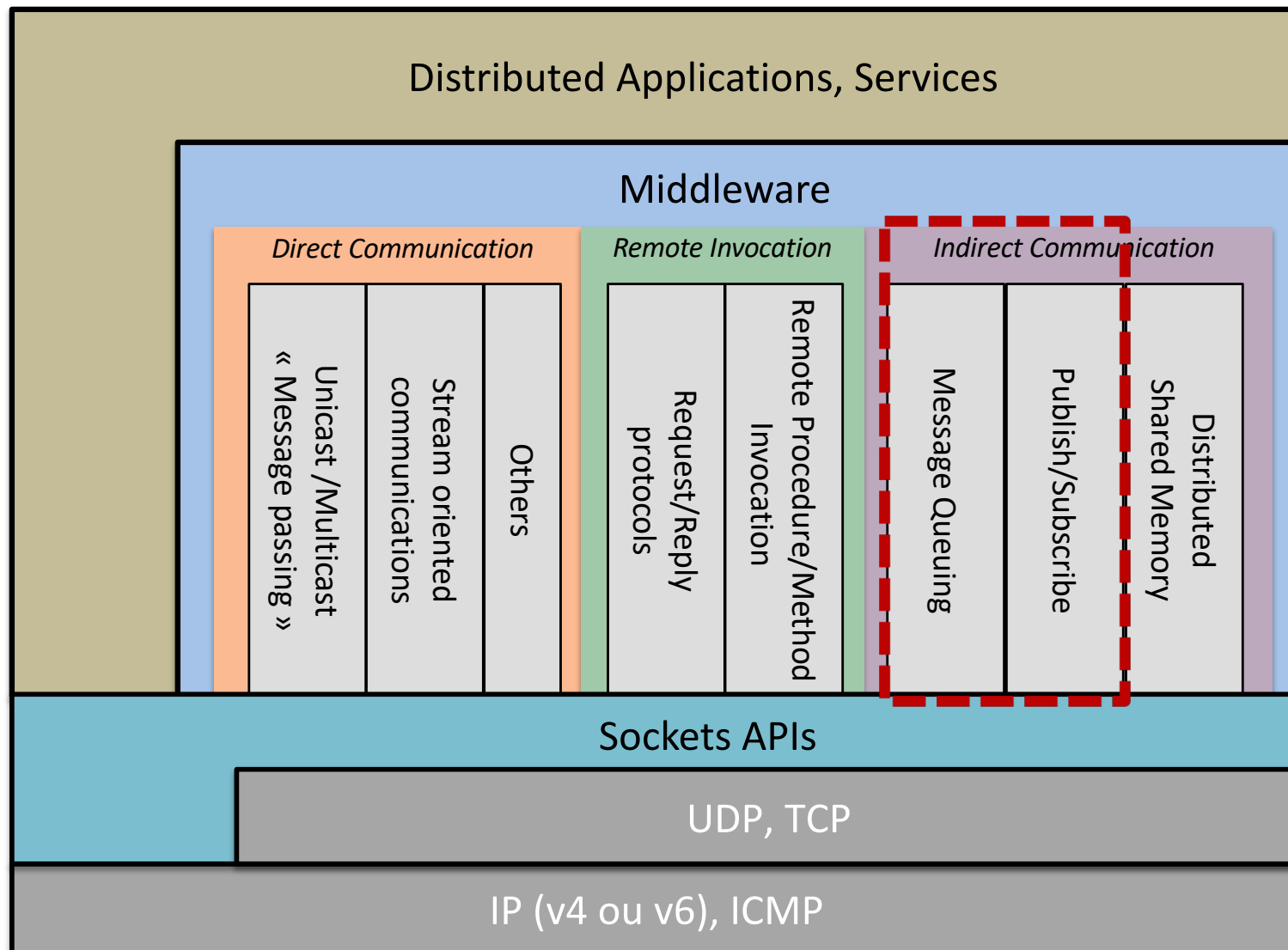
Intergiciels « réseaux » : classification



IoT : Intergiciels de *Messaging*

2 – MOMs : Intergiciels orientés Messages

MOMs : positionnement



Communication indirecte orientée messages

■ Définition

- Paradigme d'interaction qui met en œuvre un système intermédiaire de propagation de messages applicatifs entre entité émetteur (**rôle de producteur**) et entité récepteur (**rôle de consommateur**)
- **Communication asynchrone**
- Découplage entre producteur et consommateur
- Système intermédiaire de propagation de messages (**MOM**) :
 - généralement agnostique au contenu
 - et le plus fiable possible ...
- Les rôles sont définis relativement à une interaction donnée
 - Une entité au cours de son cycle de vie peut occuper successivement les deux rôles (ex : C/S asynchrone)
- Deux modèles généraux d'exécution, souvent conjointement supportés au sein d'un même MOM
 - **Message Queuing** (plutôt 1-1)
 - **Publish/Subscribe** (plutôt 1-N)

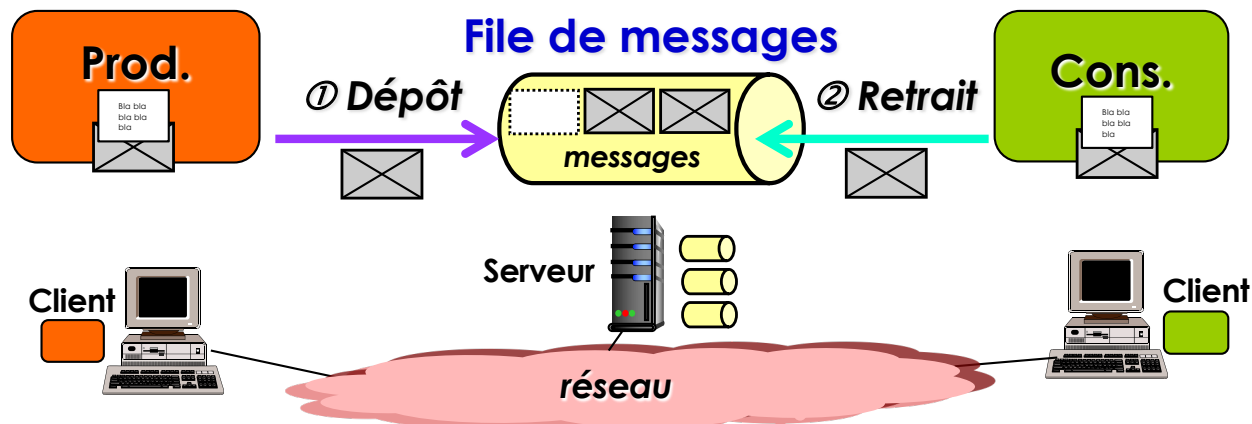
Files de messages (*message queuing*)

- **Transfert indirect de messages applicatifs**

Producteur : dépôt dans la file / Consommateur : retrait de la file

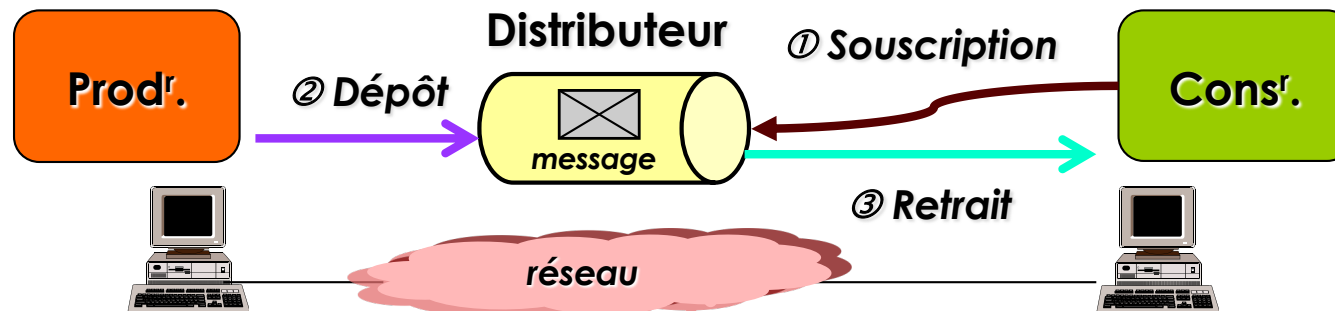
File : entité à désigner, persistance des messages jusqu'au retrait, cohérence et sécurisation

- Asynchronisme total autorisé,
- Indépendance entre le Producteur et le Consommateur (1..1 par défaut)
- Anonymat possible des entités,
- Mode « **Pull** » (défaut) : le retrait est à l'initiative du consommateur



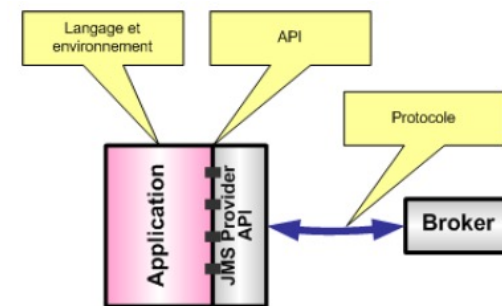
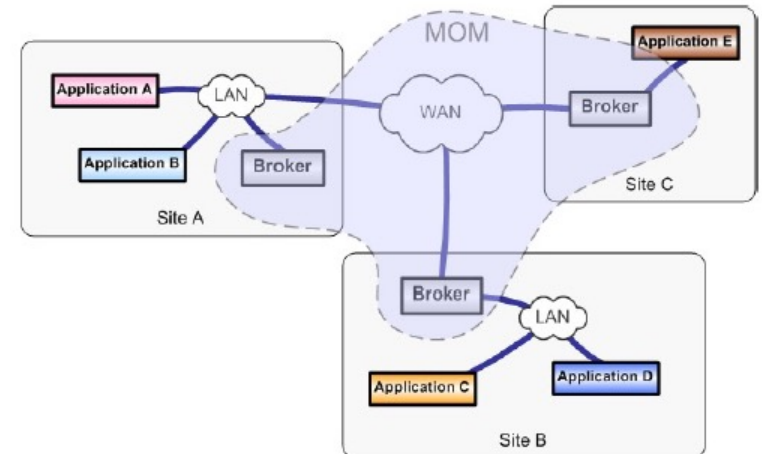
Abonnements (*Publish/Subscribe*)

- **Transfert indirect de messages applicatifs** ou d'évènements via un service intermédiaire de distribution.
- Le récepteur s'abonne (à un sujet ou un contenu)
- L'émetteur envoie un message (caractérisé ou non par un sujet) au distributeur qui le relaie à tous les abonnés concernés.
- Fort découplage :
 - Asynchronisme
 - Désignation anonyme
- Communication basique 1-N (plusieurs consommateurs peuvent s'abonner)
- Mode « **Push** » (défaut) : le retrait est à l'initiative du producteur



Services, API et protocoles

- Un **Broker** est un programme du MOM opérant un service de gestion du flux de messages :
 - On parle aussi de « provider », de « hub », de « monitor », d' « agent »
- Un Broker a un comportement de serveur logiciel : il est en attente de requêtes émanant de ses applications clientes
- Un MOM est composé d'un ou de plusieurs brokers
- Par conséquent, un « client » d'un broker peut être :
 - Un processus applicatif « producteur » ou « consommateur »
 - Un autre broker
- Leurs échanges sont soumis à un protocole (commandes invoquées et arguments, représentation des messages)
- Les protocoles sont généralement invisibles pour les applications qui ne voient que des **APIs** d'un GPL pour invoquer les opérations



Middleware orientés messages

- « Standards »

- JMS (API Java Message Service) – Sun/Oracle
- AMQP (Advanced Message Queuing Protocol) – OASIS/ISO
- CORBA Notification Service – OMG
- DDS (Data Distribution Service) – OMG
- MQTT (Message Queuing Telemetry Transport) – OASIS/ISO
- ...

- Exemples de solutions :



- MOM : briques de base des intergiciels de type EAI, ESB, DEBS...

MoM « Pub/Sub »

- Différentes variantes (...et qualité) du service rendu...
- Différente **granularité** de description et manipulation des messages :
 - **Topics-based**
 - Vocabulaire déterminé pour décrire les messages/événements
= = > utiliser un espace de nommage (plat ou hiérarchique)
 - Mise en œuvre possiblement par des files, voire interfaces dédiées
 - Expressivité limitée à la portée du vocabulaire
 - **Type-based**
 - Assez similaire au précédent : le type est un attribut du message qui est discriminant pour l'abonnement (exemple : types d'évènements)
 - **Content-based**
 - Les messages sont dits « self-contained »
 - Davantage de flexibilité... à contenir par de la sémantique (ontologies)
 - Complexité d'implémentation (filtrage et routage côté broker) et impact sur les performances

MoM « Pub/Sub »

- Différentes **topologies** de systèmes « Pub/Sub »
 - Topologie **centralisée**
 - Architecture en bus avec un service centralisé (ex : Notification Service CORBA, DDS)
 - Architecture en hub-and-spoke (ex IBM MQ)
 - Topologie **décentralisée**
 - Mécanismes de diffusion (fan out), d'inondation plus ou moins sélective
 - Nécessite parfois la mise en place de filtrage chez chaque abonné pour qu'il ne consomme que ce qui l'intéresse
 - == > possibilité de bâtir des solutions de routage « intelligent »

MoM « Pub/Sub »

- Différentes modalités de **filtrage** de l'information
 - Au niveau **global** :
 - Le broker permet la spécification de filtres lors de l'abonnement (ex : prédicat portant sur un intervalle possible de valeurs)
 - Le broker opère une vérification des prédicats avant de relayer l'information au souscripteur
 - Au niveau **local** :
 - Le récepteur abonné est responsable d'appliquer localement les prédicats de filtrage
- Différentes modalités de **distribution**
 - Mode **push**
 - Mode **pull**

MoM « Pub/Sub »

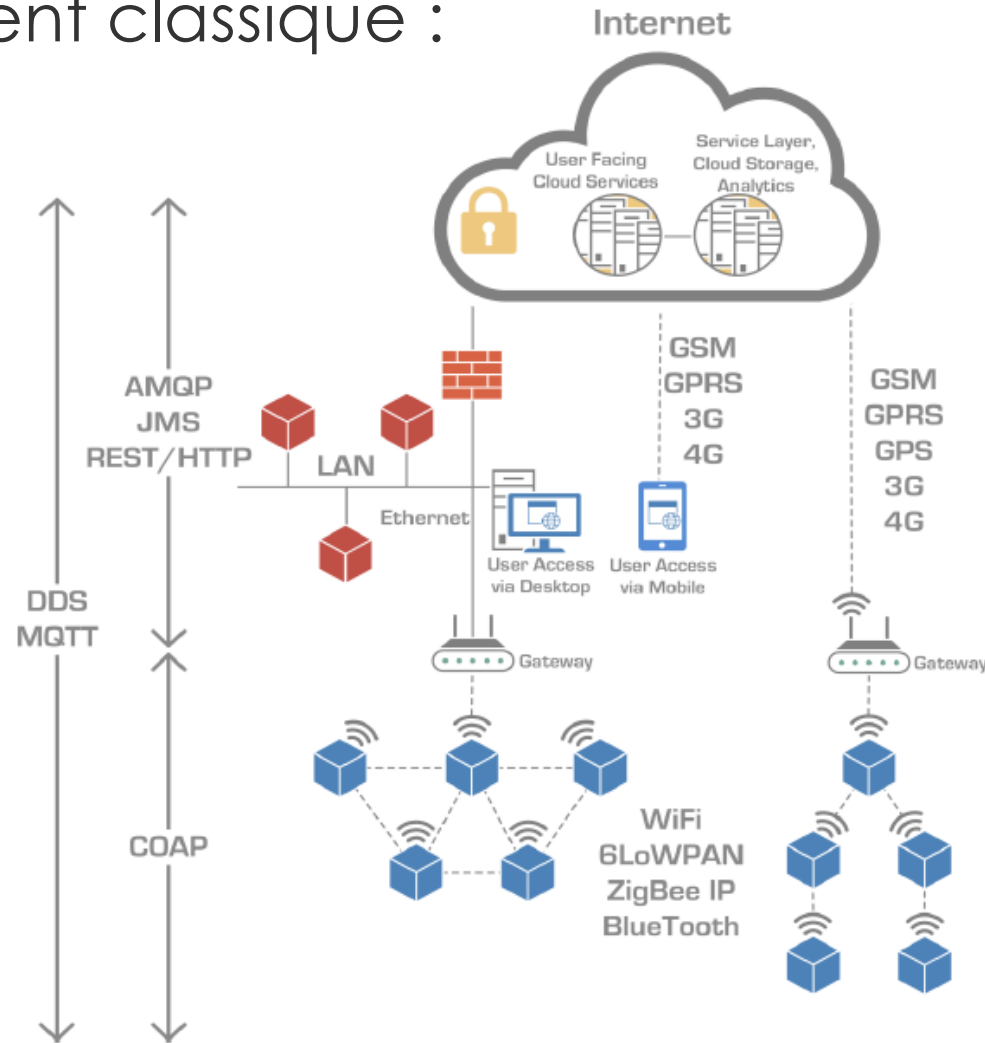
- Différentes modalités de **persistance** de l'information
 - Sans persistance :
 - Les messages publiés sont immédiatement délivrés aux souscripteurs
 - Asynchronisme autorisé :
 - non délivrance possible à des abonnés « non présents »
 - non délivrance de messages déjà publiés à de futurs abonnés
 - Avec persistance :
 - Le(s) broker(s) stocke(nt) localement les messages publiés
 - Récupération d'archives possible pour les nouveaux souscripteurs
 - La modalité de gestion de la persistance est déterminée par le producteur sous la forme d'une propriété
- Et aussi...
 - mode **transactionnel** possible : fiabilité garantie par le MOM (persistance et acquittement sur le traitement correct du message)
 - délivrance 1-1 ciblée (load balancing)
 - éléments de sécurité

IoT : Intergiciels de *Messaging*

3 – Aperçu des principales solutions dans les éco-systèmes de l'IoT

IoT infrastructure : Middleware layer

■ Positionnement classique :



IoT infrastructure : Middleware layer

- Paradigme orienté message privilégié :
 - Modèle « Pub/Sub », plus ou moins sophistiqué, **prédomine**
 - Modèle « Request/Response » subsiste également...
- Principaux protocoles standards de communication :
 - Protocoles plutôt spécifiques :
 - **MQTT** (*Message Queuing Telemetry Transport*) (Standard OASIS)
 - **CoAP** (*Constrained Application Protocol*) (IETF RFC 7252, 7959)
 - Protocoles plus généraux :
 - **AMQP** (*Advanced Message Queuing Protocol*) (Standard OASIS et ISO/IEC 19464))
 - **XMPP** (*Extensible Messaging and Presence Protocol*) (IETF 6120-22)
 - **DDS** (*Data Distribution Service*) (Standard OMG)
 - **HTTP, REST...**
- Protocoles propriétaires

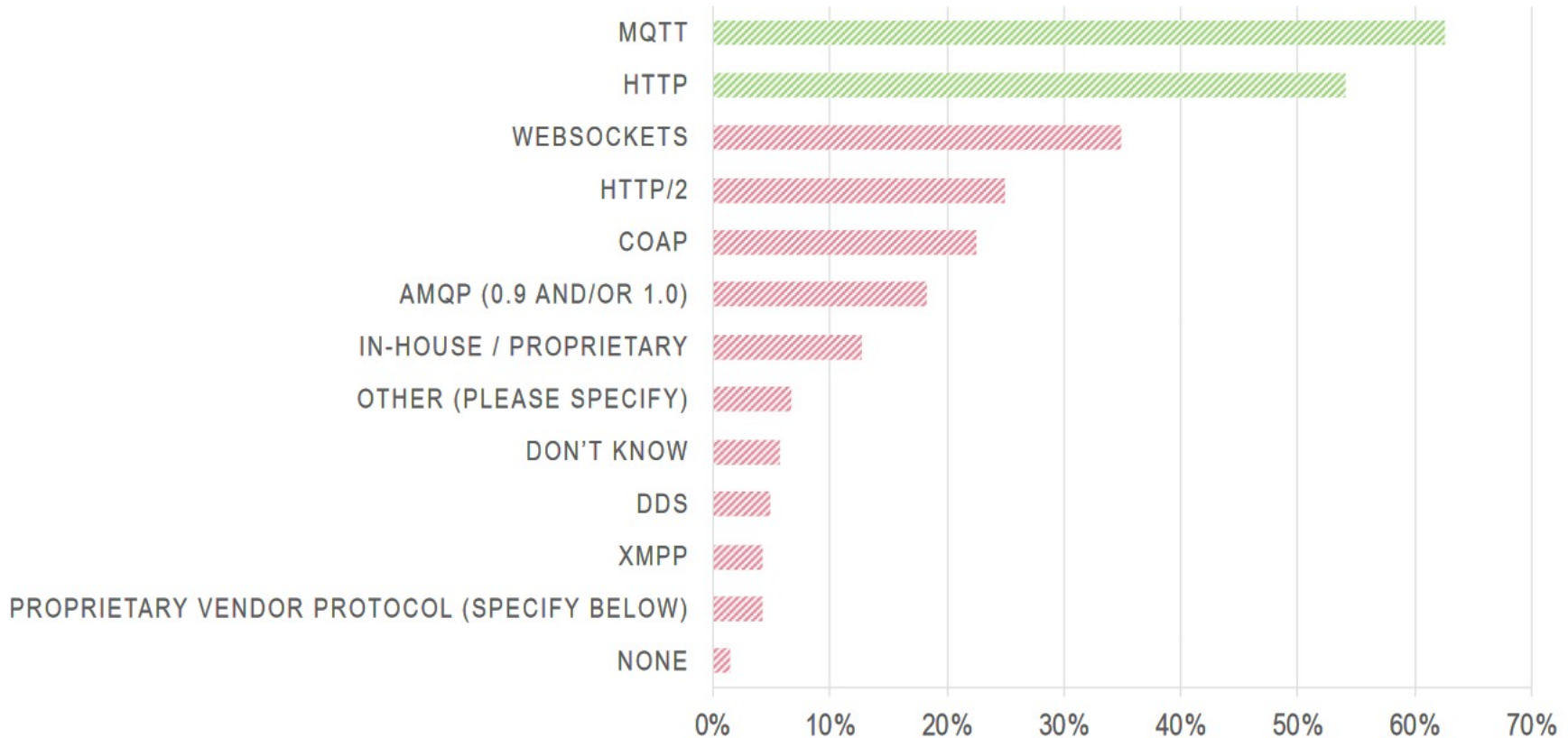
IoT infrastructure : Middleware layer

- Répartition début 2018 :

MESSAGING STANDARDS

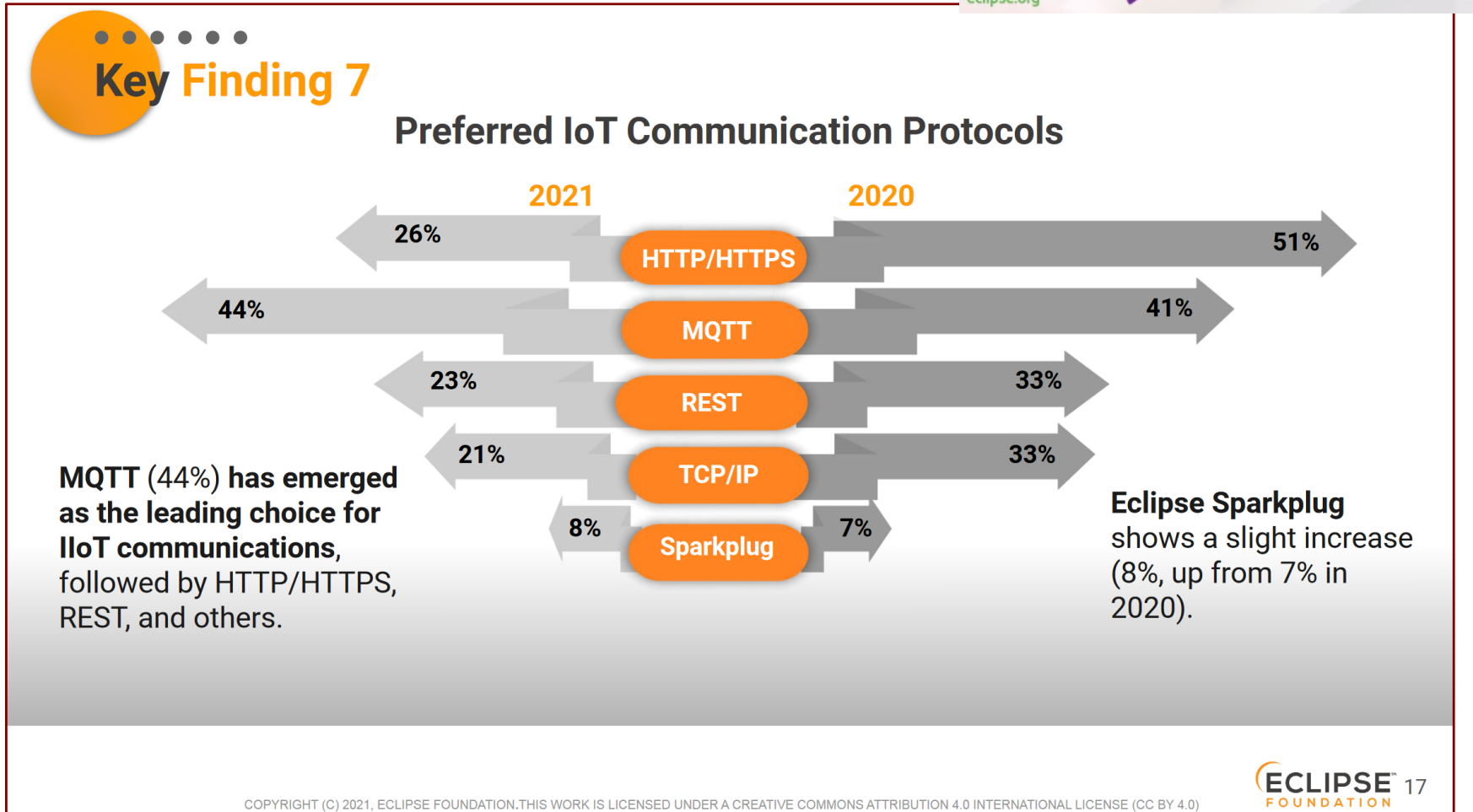


What messaging protocol(s) do you use for your IoT solution?



IoT infrastructure : Middleware layer

■ Répartition 2021 :



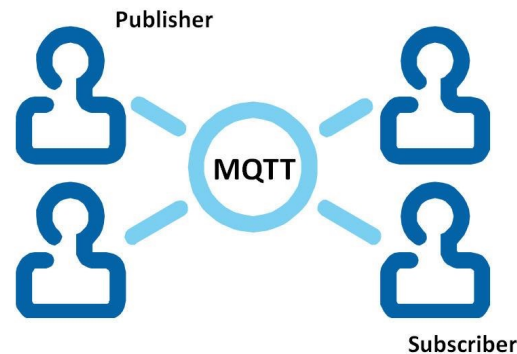
IoT : Intergiciels de *Messaging*

4 – Le protocole MQTT

MQTT : caractéristiques

■ *MQ Telemetry Transport* :

- Protocole de Transport de messages, au dessus de TCP/IP, selon un modèle de communication « pub/sub »



<http://slides.com/disk91/mqtt#/2>

- Ambition :
 - Protocole **léger**, **ouvert**, **simple** et **facile** à mettre en œuvre
 - == > adapté aux environnements contraints :
 - Empreinte de code légère (contrôleurs 8bits-256KB ram)
 - Distribution 1-N efficace de petits paquets (< 256 Mo)
 - Réseaux à faible bande passante, à forte latence, à disponibilité variable
 - == > propice : IoT, M2M, systèmes embarqués, réseaux sociaux

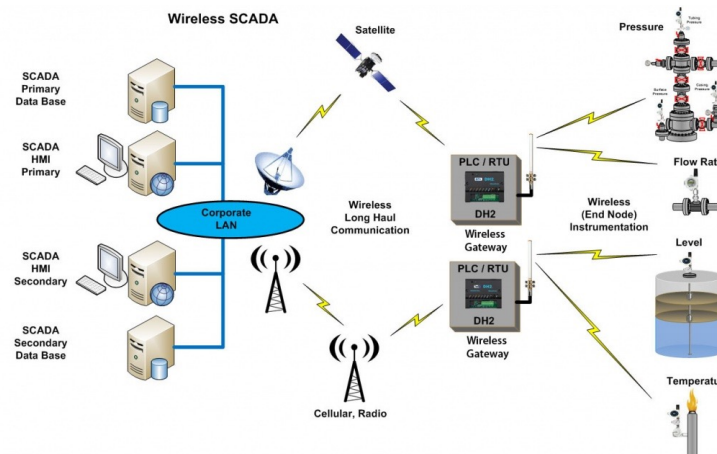
MQTT : caractéristiques

■ MQ Telemetry Transport :

■ Historique :

- Développé par **IBM** pour l'informatique pervasive industrielle (SCADA¹)

1 : Supervisory Control And Data Acquisition



<http://www.oleumtech.com/wp-content/uploads/2015/12/wireless-scada.pdf>

- Après deux années - - > communauté **Open Source**
- **MQTT v3.1.1** : standard de l'**OASIS** (soumis en 2013)
- Puis normalisation **ISO/IEC 20922:2016** (juin 2016)
- Actuellement **MQTT v5.0** standard **OASIS** (mars 2019)
- Variante pour les réseaux de capteurs **MQTT-SN** (2014)

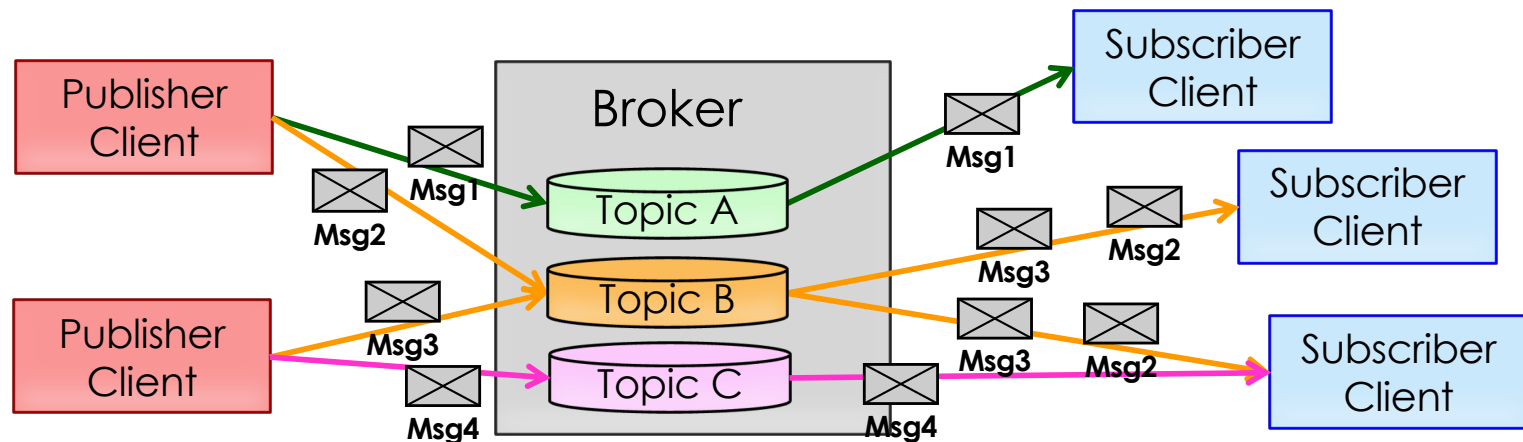


MQTT : caractéristiques

■ *MQ Telemetry Transport* :

■ Architecture :

- Modèle de « Publication/Souscription » (**Pub/Sub**) de messages à des **topics**
- Les **clients** se connectent via TCP (port 1883 par défaut) au **broker** pour ensuite publier ou s'abonner ou se désabonner sur/à un ou plusieurs topics (interconnexion de brokers possible).
- Le broker et MQTT agissent comme une interface commune et simple de connexion : **intégration facilitée et ouverte**



MQTT : caractéristiques

■ *MQ Telemetry Transport* :

■ Topics :

- **Nommage hiérarchique** structuré en arborescence :
« topic levels » séparés par ' / '
- Chaînes UTF-8, case sensitive, espaces autorisés
- Caractères génériques pour remplacer un seul niveau ('+') ou plusieurs niveaux ('#') pour la souscription
- Souscription **flexible** par filtres : si un client souscrit à :
 - "UT3PS/FSI/Indoor/#", il recevra les messages publiés avec les noms de topics suivants :
 - "UT3PS/FSI/Indoor/U3"
 - "UT3PS/FSI/Indoor/U6/temp"
 - "UT3PS/FSI/Indoor/U3/S305/temp"
 - "rugby/top14/+/ranking", il recevra :
 - "rugby/top14/StadeToulousain/ranking"
 - "rugby/top14/StadeRochelais/ranking"
 - "#", il recevra tous les messages de tous les topics

MQTT : caractéristiques

■ *MQ Telemetry Transport* :

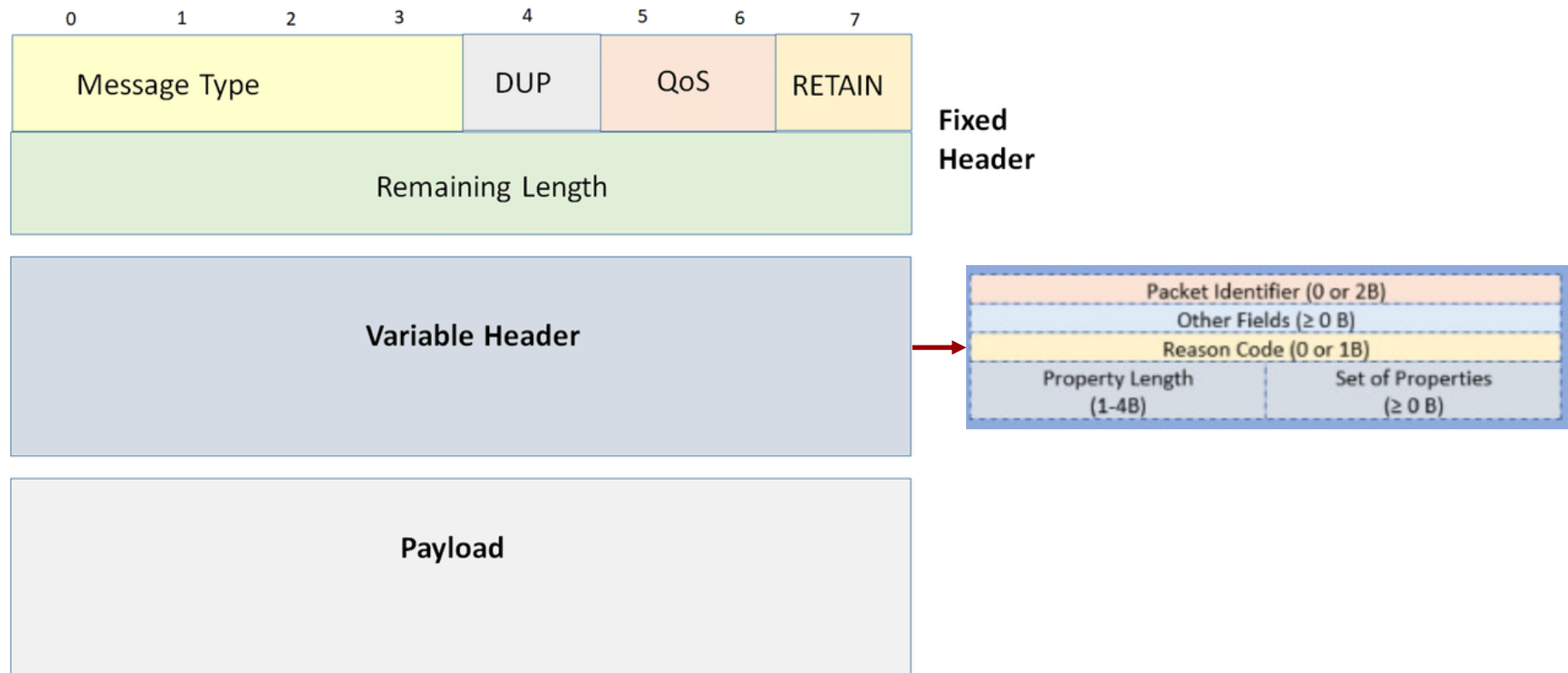
■ **Topics** : remarques et **bonnes pratiques** :

- Un broker accepte tout topic valide sans initialisation préalable
- Ne pas débiter par un ' / ', ni par un ' \$ '
- Eviter les espaces : difficultés de debug
- N'utiliser que des caractères ASCII, éviter les non éditables
- Utiliser des noms simples et concis
- Ne pas souscrire à '#'
- Penser à l'extensibilité : rajouter facilement une sous-arborescence
- Utiliser des topics spécifiques plutôt que généraux : utiliser un topic unique est un anti-pattern
- Si besoin, inclure un ID unique du publiaut

MQTT : caractéristiques

■ MQ Telemetry Transport :

■ Format général des messages MQTT :



https://www.researchgate.net/figure/Structure-of-an-MQTT-message_fig2_328606808 [accessed 27 Sep, 2022]

■ Formats spécifiques à chaque type de message

MQTT : caractéristiques

■ *MQ Telemetry Transport* :

■ Typologie des messages de contrôle MQTT :

■ **Gestion de la connexion** :

- CONNECT (C→S) CONNACK (S→C)
- DISCONNECT
- PINGREQ (C→S) PINGRESP (S→C)
- AUTH (V 5.0)

■ **Souscription** :

- SUBSCRIBE (C→S) SUBACK (S→C)
- UNSUBSCRIBE (C→S) UNSUBACK (S→C)

■ **Publication** :

- PUBLISH
- PUBACK (QoS1)
- PUBREC PUBREL PUBCOMP (QoS2)

■ Introduction de **codes de résultats d'opération** en v 5.0

MQTT : caractéristiques

■ MQ Telemetry Transport :

- **QoS** (Quality of Service) : concerne la garantie de la remise d'un message entre 1 émetteur(E) et 1 récepteur(R).

- 3 niveaux de QoS spécifiés :

- QoS0 : « **at most once delivery** » (≤ 1)

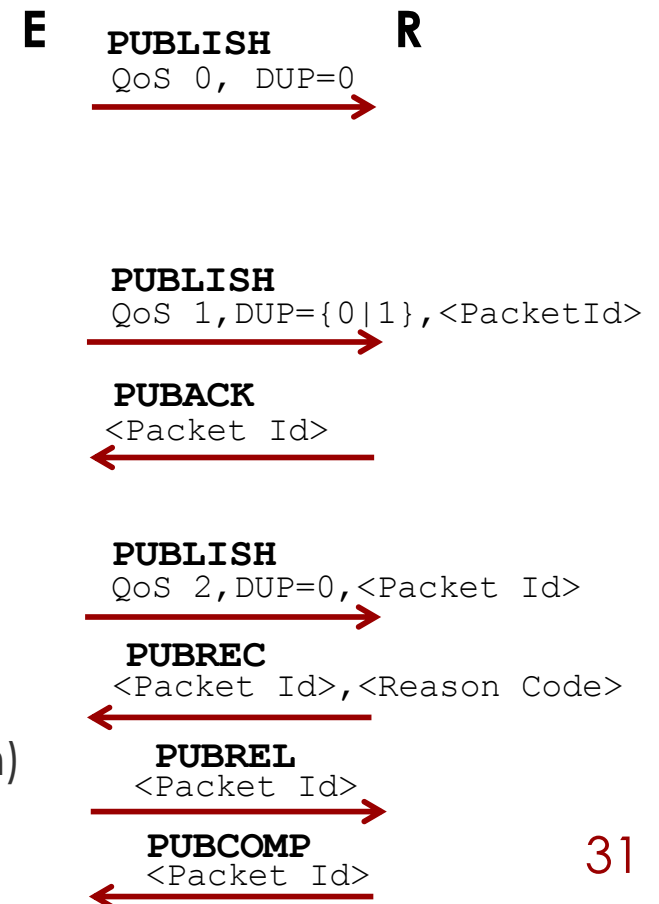
- Ni acquittement(R), ni retransmission(E) :
message reçu une fois ou pas
==> messages répétitifs et non critiques,

- QoS1 : « **at least once delivery** » (≥ 1)

- Message livré au moins une fois :
identifiant pour acquittement
et retransmission(s)
==> Importance de chaque message

- QoS2 : « **exactly once delivery** » ($\equiv 1$)

- Ni perte, ni duplication :
procédure d'acquiescement
en deux étapes (réception, compréhension)
==> overhead (bandwidth, delay)



MQTT : caractéristiques

■ *MQ Telemetry Transport* :

■ Persistance des sessions :

- **Session MQTT** : une session correspond à l'ensemble des souscriptions faites par un client ainsi que les files de messages associées pour une QoS1 ou QoS2.
(ex : messages envoyés mais non encore acquittés)
- **Gestion de session** : un client peut contrôler comment le broker gère sa session : le flag « clean session » positionné à **0** entraîne la **persistance de la session** de ce client qui sera restaurée lors de sa reconnexion. Dans le cas contraire, toutes les souscriptions du client sont supprimées dès sa déconnexion.

En V5.0, le bit CleanStart et le champ Session Expiry Interval se combinent pour simplifier la gestion de la persistance des sessions

MQTT : caractéristiques

■ **MQ Telemetry Transport :**

■ Persistance des messages :

- **Messages retenus** : un message ainsi marqué est conservé par le broker même après sa distribution vers les souscripteurs courants. Si une nouvelle souscription est enregistrée et que le message correspond au topic, le message sera envoyé au nouveau client.
- Mécanisme intéressant en cas de publications peu fréquentes.

■ Message testamentaire :

- **Last Will and Testament** : un message retenu relatif à un topic particulier est spécifié au broker par un client. En cas de déconnexion abrupte de ce client, ce message est distribué aux souscripteurs du topic. En cas de déconnexion normale, le message LWT est détruit par le broker.

■ Souscriptions partagées à partir de v 5.0

- un abonné reçoit alternativement un message de ce topic partagé
- `$share/{ShareName}/{filter}`
`$share/my-shared-subscriber-group/myhome/groundfloor/+/temperature`

MQTT : caractéristiques

■ **MQ Telemetry Transport :**

- **Sécurité** : répondre possiblement aux besoins suivants :
 - L'authentification d'équipements et d'utilisateurs
 - L'autorisation d'accéder aux ressources du serveur (topics)
 - L'intégrité et la confidentialité des paquets de contrôle et des données échangés
- MQTT n'est concerné que par la transmission des messages. Echange de login/passwd possible.
- 3 profils de sécurité :
 - L0 : « **en clair** »
 - Aucun mécanisme de sécurité additionnel
 - L1 : « **sécurité de niveau Réseau** »
 - Échange sur un réseau physique ou virtuel présentant des contrôles (ex VPN)
 - L2 : « **sécurité au niveau Transport** »
 - Utilisation de TLS : authentification, intégrité et *privacy*

MQTT : caractéristiques

- **Liste exhaustive : <https://mqtt.org/software/>**

- Exemples d'implémentations non commerciales **de brokers MQTT** :

- Mosquitto



- eMQTT

- ...sous forme de plugins

- ActiveMQ



- JoramMQ

- RabbitMQ

- etc..

- ... ou de SaaS (gratuité pour usage minimaliste)

- ThingMQ, ThingStudio, cloudMQTT, etc...

- **Nombreuses APIs pour les clients MQTT :**

- La plupart des langages

- La plupart des OS



MQTT : caractéristiques

■ **Exemple Client Pub MQTT via API Paho synchrone :**

<https://eclipse.org/paho/clients/java/>

```
import org.eclipse.paho.client.mqttv3.MqttClient;
import org.eclipse.paho.client.mqttv3.MqttConnectOptions;
import org.eclipse.paho.client.mqttv3.MqttException;
import org.eclipse.paho.client.mqttv3.MqttMessage;
import org.eclipse.paho.client.mqttv3.persist.MemoryPersistence;

public class MqttPublishSample {

    public static void main(String[] args) {

        String topic          = "MQTT Examples";
        String content        = "Message from MqttPublishSample";
        int qos               = 2;
        String broker         = "tcp://iot.eclipse.org:1883";
        String clientId       = "JavaSample";
        MemoryPersistence persistence = new MemoryPersistence();
```

.../...

MQTT : caractéristiques

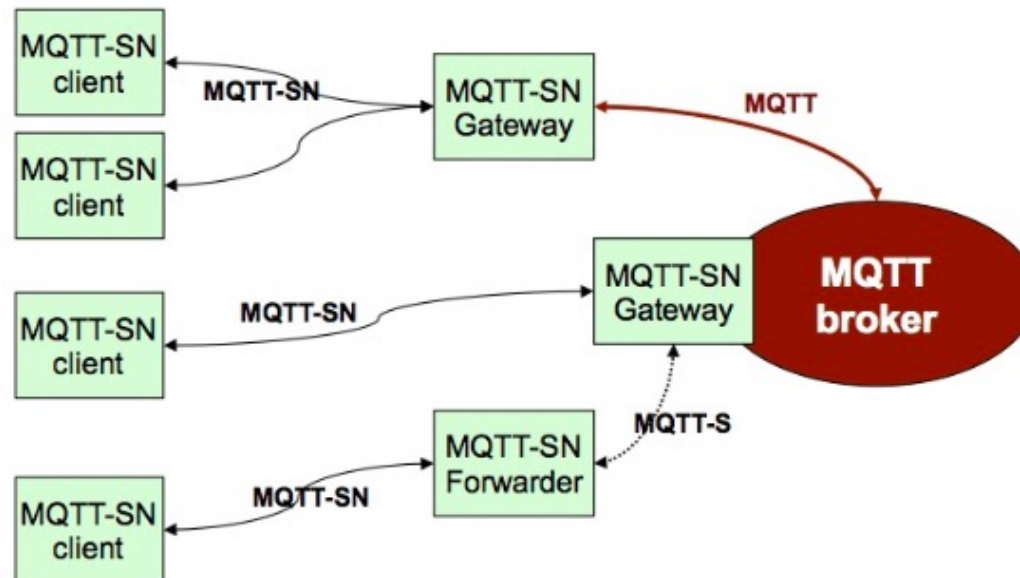
```
.../...    try {
            MqttClient sampleClient = new MqttClient(broker, clientId,
                                                    persistence);

            MqttConnectOptions connOpts = new MqttConnectOptions();
            connOpts.setCleanSession(true);
            System.out.println("Connecting to broker: "+broker);
            sampleClient.connect(connOpts);
            System.out.println("Connected");
            System.out.println("Publishing message: "+content);
            MqttMessage message = new MqttMessage(content.getBytes());
            message.setQos(qos);
            sampleClient.publish(topic, message);
            System.out.println("Message published");
            sampleClient.disconnect();
            System.out.println("Disconnected");
            System.exit(0);
        } catch (MqttException me) {
            System.out.println("reason "+me.getReasonCode());
            System.out.println("msg "+me.getMessage());
            System.out.println("loc "+me.getLocalizedMessage());
            System.out.println("cause "+me.getCause());
            System.out.println("excep "+me);
            me.printStackTrace();
        }
    }
```

MQTT-SN : caractéristiques

■ MQTT-SN

- Communication sur **UDP/IP** ou sur **non IP** (ZigBee, Bluetooth, Radio Frequency) de messages n'excédant pas 128 octets
- Conçu pour être très similaire à MQTT
- Architecture :



- Niveau de QoS supplémentaire « Simple Client » (-1) :
 - uniquement transmission (pas d'établissement de connexion)

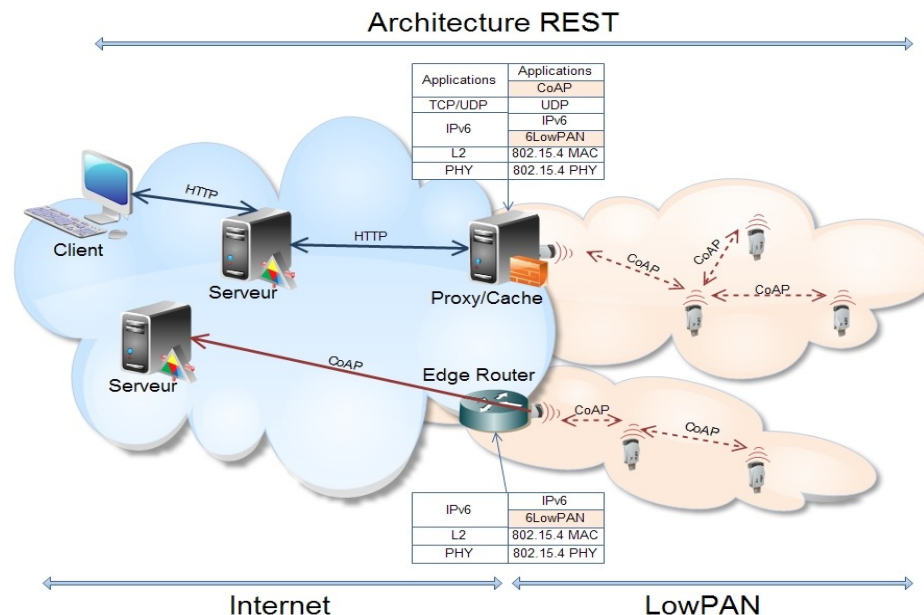
IoT : Intergiciels de *Messaging*

5 – Le protocole CoAP

CoAP : caractéristiques

■ CoAP :

- Protocole de transfert de la couche Application, orienté web, utilisable par des équipements et sur des réseaux contraints. Au dessus de UDP, il propose un modèle d'interaction 1-1(N) asynchrone « request/report » selon un style architectural RESTful
- Ambition :
 - Etendre les web services aux environnements contraints
 - ==> propice : WSNs, IoT et M2M



CoAP : caractéristiques

■ CoAP :

■ Historique :

- Issu du groupe de travail « Constrained RESTful Environments » (CORE) de l'**IETF** dans la poursuite du groupe 6LoWPANs



- 2009-2014 : spécification et évolution du protocole
- Juin 2014 : CoAP fait l'objet de la **RFC 7252**

(Z. Shelby (Sensinode), K. Hartke, C. Bormann (Universitaet Bremen TZI))

■ Extensions :

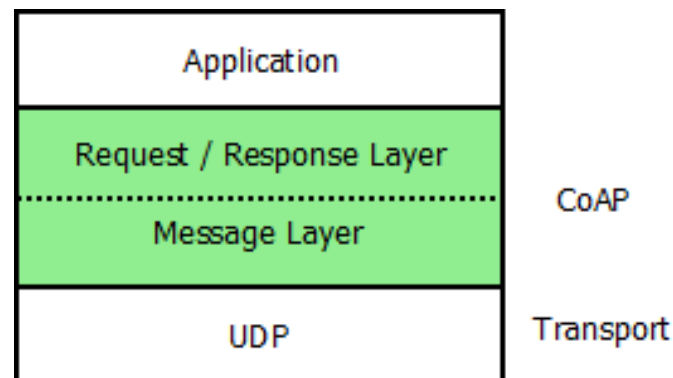
- RFC 7390 (Oct. 2014) pour la communication en multicast
- RFC 7641 (Sept. 2015) pour l'observation de ressources
- RFC 7959 (Août 2016) pour la gestion du transfert des multiples blocs d'information en multiples paires « req/rep ».
- Draft (Fevrier 2017) pour CoAP sur SMS
- Draft (Oct. 2016) pour un broker « Pub/Sub » dans CoAP

CoAP : caractéristiques

■ CoAP :

■ Architecture :

- Modèle proche de celui de http : protocole **Request/Report**
GET, POST, PUT, DELETE / CODES Réponses de type asynchrone
- CoAP est composé de deux parties :
 - Transport sur **UDP** (et sur **DTLS** pour sécurité) avec des extensions optionnelles pour renforcer la fiabilité (port **5683**)
 - Langage des messages (req/rep en *http-like* mais pub/sub possible grâce au mode « observe ») :
 - Messages confirmables ou pas (~QoS); Superposition req/rep REST possible dans les messages d'acquittement (*Piggybacking*)



CoAP : caractéristiques

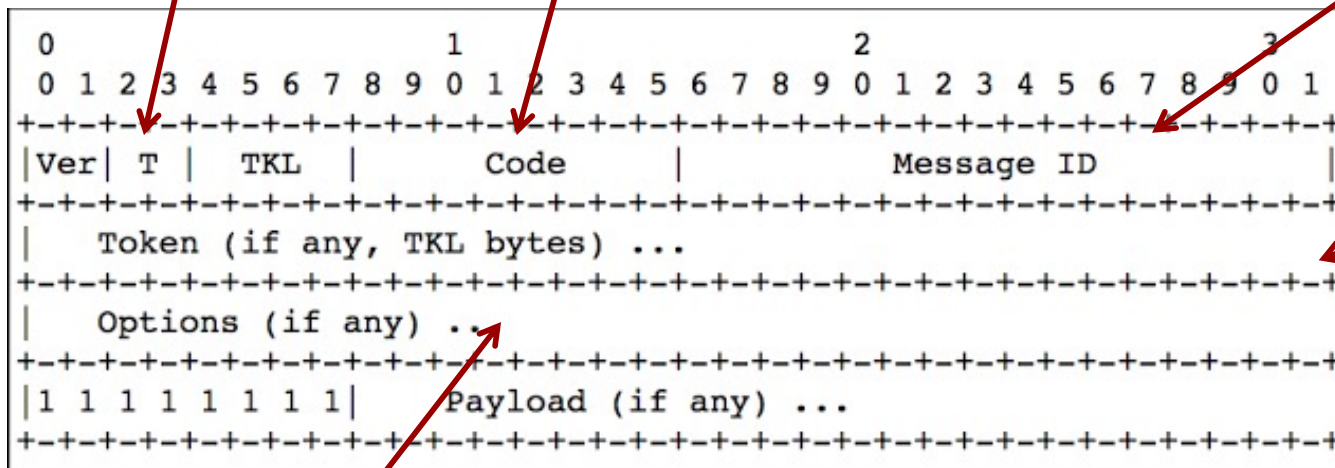
■ CoAP :

■ Messages : en-tête fixe de 4 octets

{ Confirmable, Non-Confirmable, Acknowledgement, Reset }

{Request Method (1-10) ou Response code (40-255)}

Identifiant
de mise en
correspondance
lors de
confirmation



Corrélacion
Requête/Report

Au format (T,L,V), informations optionnelles ou par défaut
d'une requête ou d'une réponse (ex : URI, type de payload...)

CoAP : caractéristiques

■ CoAP :

■ Représentation de Ressources et de services :

- CoAP utilise des URIs comme représentation standard, décomposées ensuite en Options dans les messages

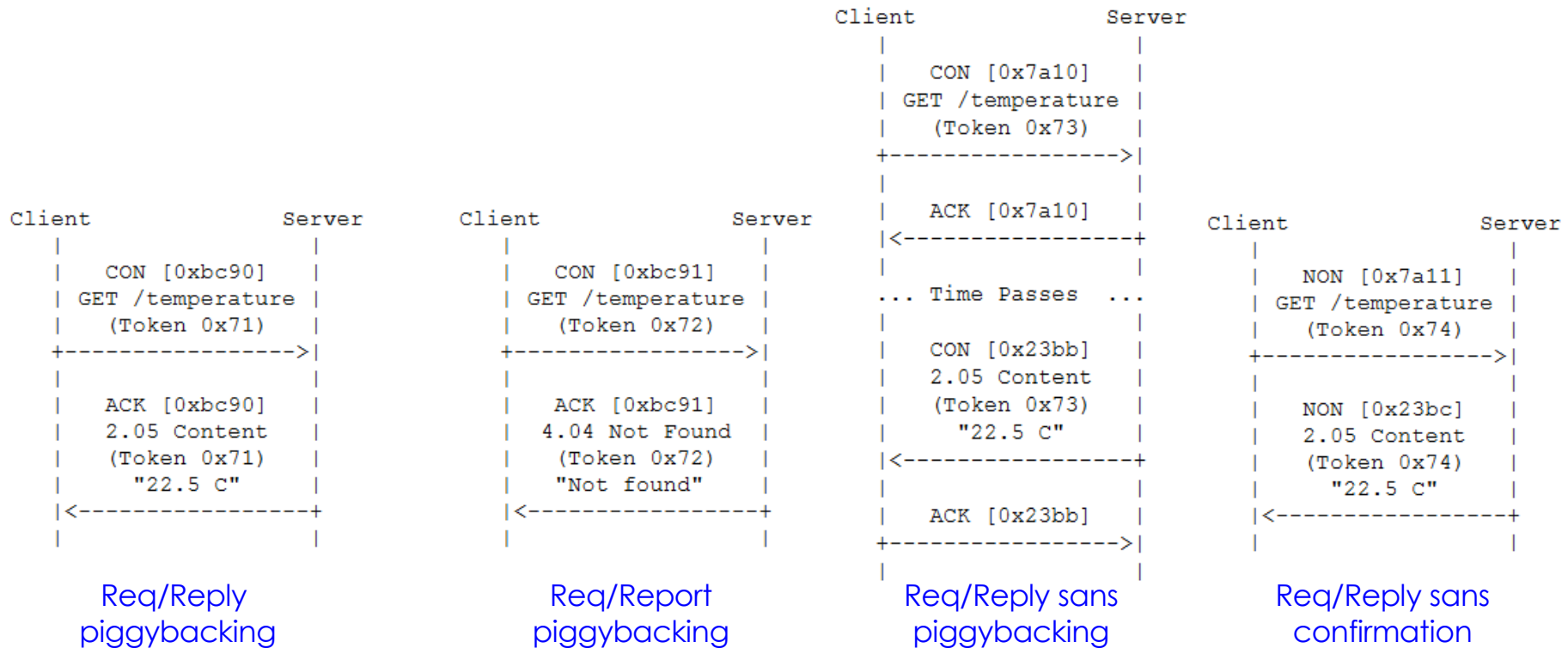
`coap-URI = "coap:" "://" host [ ":" port ] path-abempty [ "?" query ]`

`coaps-URI = "coaps:" "://" host [ ":" port ] path-abempty [ "?" query ]`

CoAP : caractéristiques

■ CoAP :

- Patterns d'interactions **Request/Reply** [RFC 7252]:
 - Requêtes/Réponses, mode *pull*, asynchrone, avec ou sans confirmation



CoAP : caractéristiques

■ CoAP :

■ Pattern d'interaction d'**observation** [RFC 7641] :

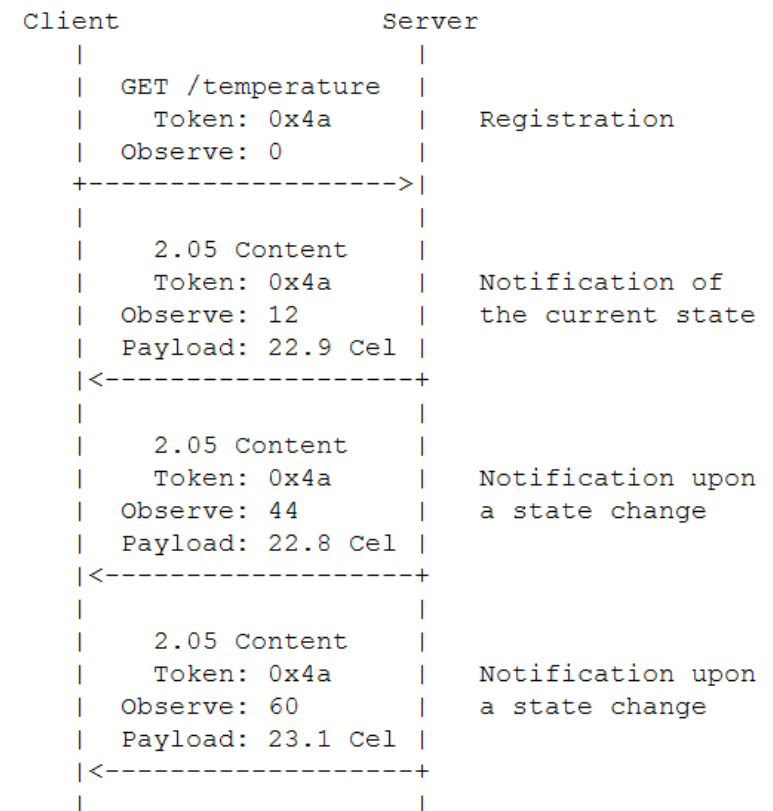
■ *pushed* notifications, confirmées ou non

■ Option Observe :

- Dans requête GET : souscription (0) ou retrait (1)
- Dans réponse : identifie le message comme une notification et la numérote

■ Le serveur peut exposer les ressources observables qui changent d'état de façon utile à l'application. Exemples :

- `<coap://server/temperature>`
envoi périodique
- `<coap://server/temperature/felt>`
dépassement de seuil(s) (chaud, froid)
- `<coap://server/temperature/critical?above=42>`
oscillation autour d'un seuil spécifié par le client



CoAP : caractéristiques

■ CoAP :

- Pattern de **transfert de blocs** [RFC 7959] :
 - Transaction bloc par bloc
 - Options Block1 Block2

```
CLIENT                                     SERVER
|                                         |
| CON [MID=1234], GET, /status, 2:0/0/64   -----> |
|                                         |
| <----- ACK [MID=1234], 2.05 Content, 2:0/1/64 |
|                                         |
| CON [MID=1235], GET, /status, 2:1/0/64   -----> |
|                                         |
| <----- ACK [MID=1235], 2.05 Content, 2:1/1/64 |
|                                         |
| :                                         :
| :                                         :
| :                                         :
| :                                         :
| CON [MID=1238], GET, /status, 2:4/0/64   -----> |
|                                         |
| <----- ACK [MID=1238], 2.05 Content, 2:4/1/64 |
|                                         |
| CON [MID=1239], GET, /status, 2:5/0/64   -----> |
|                                         |
| <----- ACK [MID=1239], 2.05 Content, 2:5/0/64 |
|                                         |
```

CoAP : caractéristiques

■ CoAP :

- Exemples d'implémentations open source :
 - Californium et extensions (sandium, actinium): cadriciel Java
 - Erbium (C) : moteur REST pour Contiki OS
- Outils de tests et d'intégration :
 - Copper pour Firefox (Javascript)
 - Coap.me (récupération de serveurs coap)
- Outil de simulation
 - Cooja



- Pour davantage de détails :

<https://coap.space/impls.html>